

Local Computation: Genome Choice and Evolved Cellular Automata Rules

Henry LeCates, Jesse Looney, Paolo Canigiula, and Michael Allers

Amherst College, USA

{hleccates26,jlooney27,pcanigiula26,mallers26}@amherst.edu

Abstract. Michael

1 Introduction

Biological systems possess a remarkable ability to generate complex, robust global structures from simple, local interactions. From the development of a multicellular organism from a single zygote to the coordinated behavior of bacterial colonies through quorum sensing, nature excels at "local computation." In these systems, every individual unit, whether a cell or an organism, follows a universal rule based solely on its immediate environment, yet these local actions culminate in sophisticated emergent behaviors such as morphogenesis and self-repair.

The field of Cellular Automata (CA) seeks to replicate this phenomenon by defining local update rules that guide the state of each cell in a grid. Cellular automata are universal computational systems, where in principle any computable function can emerge from a collection of simple cells. Hence, there is no limit to what cellular automata systems can compute, including simple geometric patterns (e.g., stripes, checkerboards, etc), complex mathematical functions (e.g., patterns with combinatorial structure), and biological functions (e.g., self-reproducing or self-modifying configurations).

However, manually designing these rules to achieve specific global objectives is notoriously difficult. To overcome this, researchers have turned to evolutionary computation and machine learning. A prominent approach is Neural Cellular Automata (NCA), such as the CA-NEAT framework, which utilizes NeuroEvolution of Augmenting Topologies (NEAT) to discover rules represented as neural networks. The biological inspiration for NCA is well-studied: neural networks mirror gene regulatory networks and inter-neuronal signaling, in which one component continuously regulates another through weighted interactions. Therefore, this genome representation lend itself very well to expressing smooth, distributed dependencies across many inputs.

However, biological computation is not exclusively continuous. In nature cells also make discrete, state-based decisions: an operon is either expressed or not, a cell adopts one fate or another, and quorum-sensing bacteria switch collective behaviors once a chemical concentration crosses a threshold. These mechanisms operate by sharp, conditional logic rather than smooth integration. Stack based

genetic programming with Push offers a genome representation that mirrors this mode of biological computation. A Push program is a linear sequence of typed instructions built from primitives (e.g., Boolean logical operations, threshold tests, and stack manipulations). This makes it well suited to express sharp, position specific rules. The two genome representations, Neural and Push based, represent to distinct modes of biological computation. Hence a natural questions is how their evolutionary tendencies compare when applied to the same cellular automata task.

In this paper we present a novel implementation of Push-based cellular automata (PCA), a baseline implementation of neural cellular automata (NCA) following the CA-NEAT framework, and a novel hybrid representation (HyCA) that combines them: a CPPN consumes the neighborhood and produces a small number of continuous signals, which are used as additional inputs into a Push program that makes the discrete update decision. The hybrid is itself biologically motivated, mirroring how real cells combine continuous chemical signals with threshold-based decision logic. In the following sections we explore the effectiveness of all three representation

2 Preliminaries

2.1 Cellular Automata

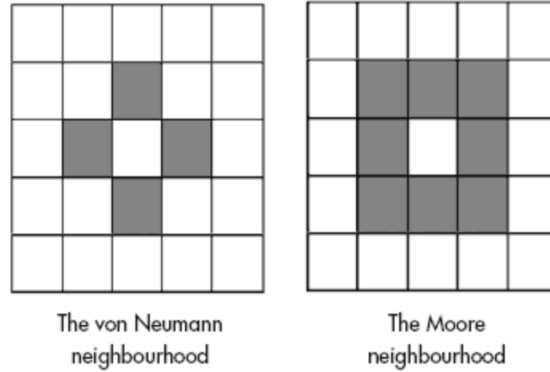


Fig. 1: Two classic CA neighborhood examples

Cellular Automata (CA) are dynamical systems consisting of a regular grid of cells, each of which exists in a finite number of states. Originally envisioned by John von Neumann in the 1940s to model biological self-reproduction, CA represent one of the simplest models of parallel computation. In their most basic one-dimensional form, the flow of information is determined by the neighborhood scope: one-way CAs update a cell's state based on itself and a single neighbor

(e.g., to the left), while two-way CAs incorporate inputs from both adjacent sides. This local interaction allows for the emergence of complex, computationally universal behaviors from a set of simple, uniform rules applied across the lattice. In two-dimensional systems, the transition function is defined by the specific geometry of a cell’s environment. The Von Neumann neighborhood is a diamond-shaped configuration that considers the four cardinal neighbors (North, South, East, and West), totaling five cells including the center (see Fig. 1). In contrast, the Moore neighborhood is more comprehensive, encompassing all eight surrounding cells in a 3×3 square. During each discrete time step, every cell in the grid updates its state simultaneously according to a global rule. This process can generate a vast range of phenomena, from stable periodic patterns like checkerboards to intricate, self-similar structures such as the Sierpinski triangle [3].

We restrict our analysis to two-dimensional binary cellular automata with a von Neumann neighborhood. With this setup, each cell takes a state in $\{0, 1\}$ and its update rule depends on the states of five cells (the cell itself and its four cardinal neighbors).

2.2 Neural Cellular Automata via NEAT

Neural Cellular automata (NCA) replaces the typically update rule with a parametrized function appropriator, typically in the form of a neural network, that maps a cell’s neighborhood values to its next state [4]. NeuroEvolution of Augmenting Topologies (NEAT) is a genetic algorithm designed to evolve both the weights and the topology of the network [8]. NEAT begins with a population of minimal networks, consisting only of the input and output nodes. Then during additional hidden layers and connections are added via structural mutations. To reserve structural mutation long enough for their weights to be mutated, NEAT tracks the origin of each structural innovation via an innovation number and groups topologically similar individuals into species, within which selection and mutation occur. This allows newly introduced structural mutations to be tuned before they are selected away.

Oftentimes the evolved network is maintained as a form of a Compositional Pattern Producing Network (CPPN). A CPPN is a network who uses a set heterogeneous activation functions to encode symmetries and repetitions found in natural development [7]. When CPPNs are applied to cellular automata, the resulting algorithm is called CA-NEAT [5]. Here a single evolved CPPN is applied is used as the update rule for all cells in the grid. Each cell’s neighborhood values (and typically normalized positional arguments) are used as input to the CPPN, and the output neurons with the highest value determines the cell’s next state. CA-NEAT has been shown to evolve solutions to 2D morphogenesis and replication tasks.

2.3 Stack-based GP with Push

Stack-based genetic programming presents itself as an alternative to classical tree-based approaches. Push is a stack-based programming language designed to be the representation for programs in evolutionary systems [6]. In Push, there are multiple stacks, one for each data type relevant to the problem at hand. Instructions pull their inputs from and push their outputs to stacks, doing nothing if enough inputs are not available or they cannot produce an output (e.g. division by zero). In the simplest case (and in our treatment here), a Push program is just a sequence of instructions executed one after another. The output of the program is the final state of the stacks, though usually a single value is pulled off the top of some stack of interest for use as output. The tolerance of Push programs to underfed or faulty instructions or early termination makes Push a particularly suitable representation for evolution.

In contrast to tree-based representations, Push represents programs with a linear genome which is easy to vary without special considerations such as type-checking or counteracting program bloat. Push programs are typically mutated using uniform mutation by addition and deletion (UMAD), which has been found to be highly effective on many genetic programming problems [2].

3 Genome Representations for Evolving CA Rules

Include the biological inspiration for each method in these sections

3.1 PCA: A Push-Based Genome

Push-based cellular automata (PCA) use Push programs for their local rules. Each individual is represented by a linear genome of Push instructions. These instructions include the neighborhood constants $N_0, \dots, N_n$ where n is the number of cells in the choice of neighborhood (e.g. $n = 5$ in the case of a radius-one von Neumann neighborhood). To determine the next value of a cell, the neighborhood constants are set to the values of the neighboring cells, and then the program is evaluated. The next value of the cell is the value on the top of the appropriate stack at the end of the program.

Since we focus on binary cellular automata, our representation has just one stack, which manipulates Boolean values. We therefore include the constants `true` and `false`, in addition to a selection of standard logical operations: `not`, `and`, `or`, `xor`, `nand`, and `nor`. In principal, subsets of these operations, such as `{not, and, or}` or simply `{nand}` are, together with the constants, enough to build all logical operations. In practice, it is helpful for evolution to have access to the "right" operations for a given problem, rather than needing to build them on its own. We did not explore in detail the tradeoff between a simpler genome and having an abundance of available operations.

3.1.1 PCA vs CA-NEAT: An Initial Comparison Paolo?

3.2 NCA: A Network-Based Genome

Neural Cellular Automata (NCA) is motivated by the idea that a cell’s update rule can be represented as a continuous function. A Boolean program (PCA) or lookup table commits to harsh discrete boundaries; however, a CPPN can learn smooth nonlinearities and create complex decision boundaries. From Section 3.1.1 we see that our baseline implementation of Push based cellular automata (PCA) fails to match the effectiveness of CA-NEAT. This is largely due to the nuanced nature of CA-NEAT. Each CPPN takes as input normalized positional arguments and neighborhood values, and there exist a complex system of speciation and innovation, allowing weights to be fully optimized. We want to restrict our PCA implementation to local computation only (i.e., no positional arguments), therefore were motivated to create a baseline comparison NCA.

Concretely, in our baseline NCA implementation, each individual is represented as a single CPPN genome evolved via NEAT. Cell updates proceed in one stage: the five Von Neumann neighborhood values (center, north, south, east, west) are each normalized from Boolean to the continuous range $[-1, 1]$ and passed as inputs to the network. The CPPN produces two output values, with the greater one determining the cell’s next state. The CPPN topology is evolved using NEAT’s three structural operators: weight perturbation, connection addition between previously unconnected acyclic nodes, and node insertion by splitting an existing connection and assigning a random activation function from a library of fourteen. Evolution is organized as speciation; however, there is no concept of innovation, instead each genome is clustered by a compatibility distance, $\delta = c_1 \cdot \frac{\text{disjoint}}{N} + c_3 \cdot \text{avg_weight_diff}$. Within each species, parents are chosen by lexica selection on a window of per-step CA errors. Species with five or more members preserve their fittest genome as an elite.

3.3 HyCA: A Hybrid Genome

Paolo could you check this biology? Going off the dome

Hybrid cellular automata (HyCA) is motivated by the biological observation that cell decisions are typically not made directly from raw spatial position. Instead cells use a mix of spatial position and various threshold reading from continuous chemical signals. Each cell makes a discrete decision by comparing local chemical signals to internal thresholds. The continuous chemical signals and the discrete decision are produced by different mechanisms. Neither mechanism is well-suited to performing the opposite task, or operating solo. HyCA mirrors this division explicitly: a CPPN plays the role of the smooth, continuous chemical input process, while a Push program plays the role of the threshold-based decision logic.

Concretely, each individual is represented as a pair $\{\text{:cppn}, \text{:program}\}$, combining a Compositional Pattern Producing Network evolved via NEAT and a Push program evolved via UMAD mutation. Cell updates follow a two-stage process. First, the CPPN receives the five Von Neumann neighborhood values and produces k continuous signals; we use $k = 3$ for the simple targets and $k = 4$

for the Sierpiński target. Second, the Push program executes with the five raw Boolean neighborhood values pushed onto the Boolean stack and the k CPPN signals pushed onto the float stack. As in PCA the top of the Boolean stack at the end of execution determines the cell’s next state.

The two components are evolved jointly under a single fitness function but with representation-specific variation operators. NEAT’s structural and weight mutations act on the CPPN, while UMAD acts on the Push program. An offspring inherits a

4 Evaluation on Evolving Target Patterns

4.1 Experimental Setup

We evaluate each method on six target patterns, spanning a range of difficulties: a checkerboard, vertical and horizontal stripes, sparse and dense dots, and wide stripes. Each target pattern is a binary 10×10 grid with a von Neumann neighborhood and wrap-around boundary conditions. Starting from a fixed initial condition, a cell at $[0, 0]$, a candidate rule is run for 30 update steps. Each step is checked to verify if it matches the target pattern.

Fitness is measure as the mean per-cell error $\text{err}(G, G^*) = \frac{1}{N} \sum_{i,j} \mathbf{1}[G_{ij} \neq G^*_{ij}]$ where G is the grid produced by the candidate rule and G^* is the target. Lower is better, with zero corresponding to an exact match.

Each method uses the same population size of 200 and is run for 50 generations on the simple targets and 200 generations on the Sierpiński target.¹ PCA evolves Push programs under UMAD mutation with lexicase parent selection, where each lexicase case is the grid produced by the rule at a single update step from some step x through the final step. This pressures rules to match the target across a range of points in their trajectory rather than only at the final step. NCA evolves CPPN-style networks via NEAT with standard parameter settings. HyCA jointly evolves a CPPN (NEAT) and a Push program (UMAD), with three continuous CPPN signals (four for the Sierpiński target) fed into the Push program alongside the five raw neighborhood values. All experiments are repeated over 10 independent runs; convergence plots show the mean best-of-generation error with shaded ± 1 standard deviation bands.

4.2 Convergence and Final Performance

Figures 2–7 show the best-error trajectories for each method on the six targets.

On the checkerboard target (Figure 2) are able to reach zero error within about 10 generations. The local rule for a checkerboard is captured almost entirely in the dynamics of the neighborhood; however, the pattern serves an efficient baseline to ensure each genome representation evolves properly.

On vertical stripes, horizontal stripes, and sparse dots (Figures 3, 4, 5), there is a clear separation of each method. PCA reaches an errors close to zero within

¹ The Sierpiński target is evolved on a 30×30 grid and evaluated over 80 update steps.

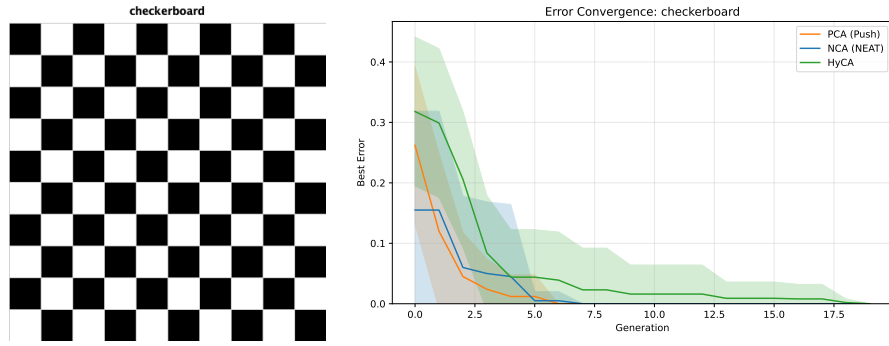


Fig. 2: Target pattern (left) and convergence plot for evolving a checkerboard using NCA, PCA, Hy-CA (right).

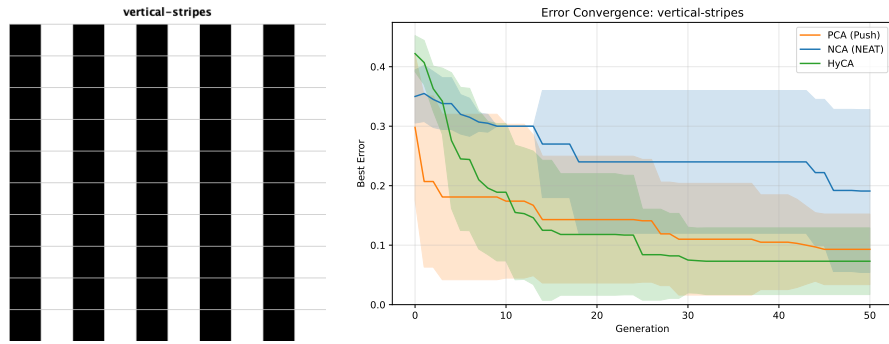


Fig. 3: Target pattern (left) and convergence plot for evolving vertical stripes using NCA, PCA, Hy-CA (right).

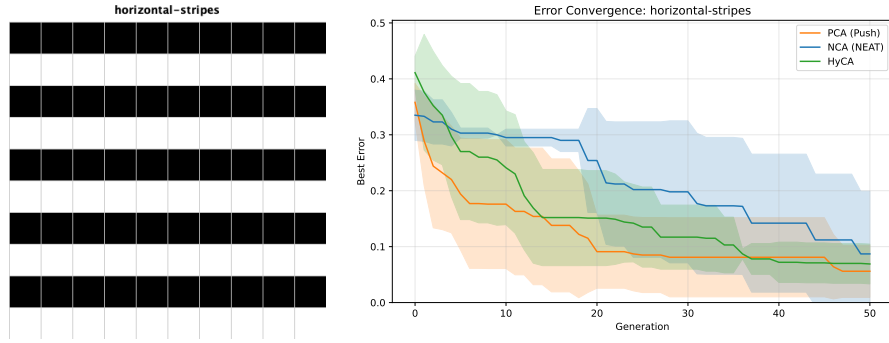


Fig. 4: Target pattern (left) and convergence plot for evolving horizontal stripes using NCA, PCA, Hy-CA (right).

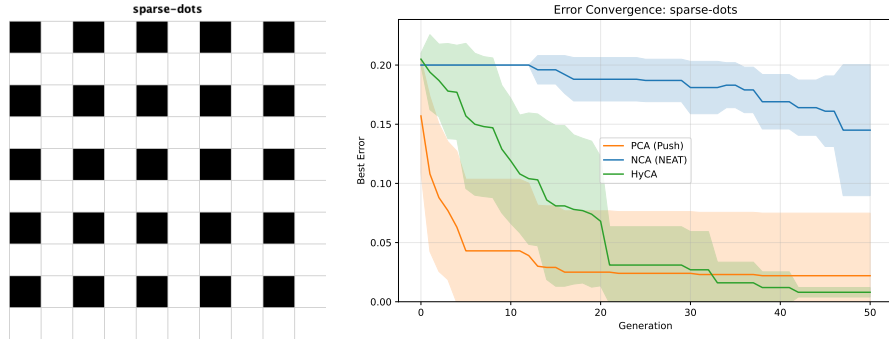


Fig. 5: Target pattern (left) and convergence plot for evolving sparse dots using NCA, PCA, Hy-CA (right).

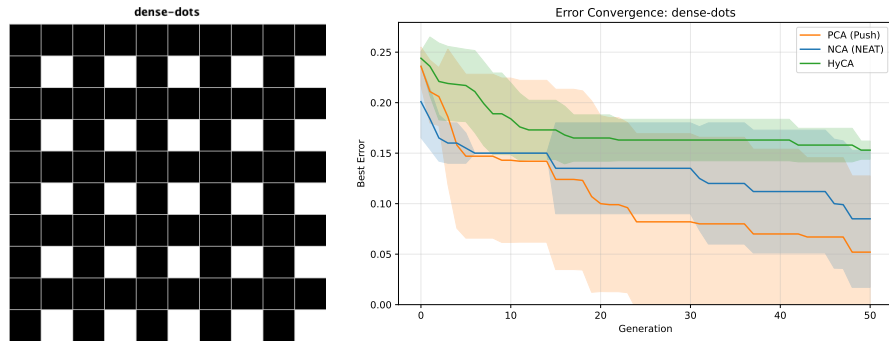


Fig. 6: Target pattern (left) and convergence plot for evolving dense dots using NCA, PCA, Hy-CA (right).

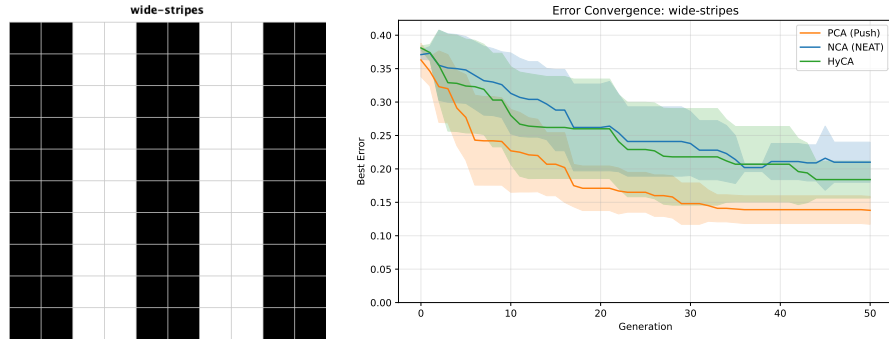


Fig. 7: Target pattern (left) and convergence plot for evolving wide strips using NCA, PCA, Hy-CA (right).

2- to 3 generations, HyCA is competitive, and NCA plateaus at a higher error (roughly 0.15–0.20). These patterns require a rule that is sharply dependent on a single coordinate of the neighborhood (e.g., copy the cell to the west). This can be more easily expressed as a discrete decision in Push’s boolean based operations, but appears to be harder for a NEAT-evolved network to recover from a single point in weight space. HyCA’s performance here suggests that once a Push program is in the loop, the CPPN component does not impede discovery of the same discrete rule.

Dense dots and wide stripes (Figures 6, 7) are harder for every method, and the ordering changes. PCA still attains the lowest error (roughly 0.05 on dense dots, 0.13 on wide stripes), but NCA matches or slightly outperforms HyCA.

Add some reasoning

Across all six simple targets, PCA reaches the lowest final error, HyCA is competitive on the more strongly periodic patterns, and NCA converges quickly when the rule is simple but has difficulty escaping a higher-error plateau on the harder targets.

4.3 Solution Complexity

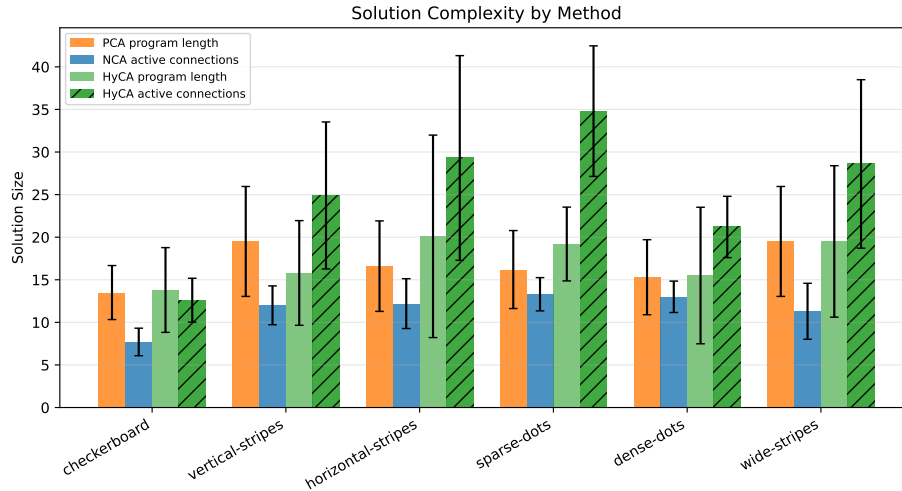


Fig. 8: Complexity plot of each genome. The complexity of PCA is shown as the length of the program. NCA complexity is shown as the number of active connections. The complexity of HyCA is shown as both number of active connections and length of program for its respective parts.

Figure 8 reports the complexity of the evolved solutions, measured for each genome in its respective units: Push program length for PCA, number of active connections for NCA, and both quantities for HyCA.

NCA produces the simplest genomes with respect to its own unit, typically using fewer than 15 connections. PCA programs tend to be longer, ranging from 13 instructions on the checkerboard to 20 instructions on the vertical strips pattern. The HyCA’s Push component tends to be a comparable length to PCA; however, its CPPN component is substantially more complex than NCA, with more active connections on all patterns. Hence, the HyCA complexity exceeds both PCA and NCA on all target patterns.

This is consistent with the convergence results. HyCA takes advantage of the both representation expressiveness but it must explore a much larger evolutionary search space. Furthermore, on the six simple patterns, it may be the case that the CPPN input is essentially a no-op once a working Push program has been found. This motivates the following investigation of each genome representations effectiveness on a more complex pattern, the Sierpiński triangle.

4.4 Performance on Complex Patterns

To investigate the limits of each representation we evaluate on the Sierpiński triangle (Figure 9), a fractal whose generating rule (XOR of two neighbors, under suitable boundary conditions) is a classic example of complex behavior produced by a simple local rule. Unlike the simple periodic targets, a correct rule must construct the fractal from the seed and then leave it alone, no longer updating it.

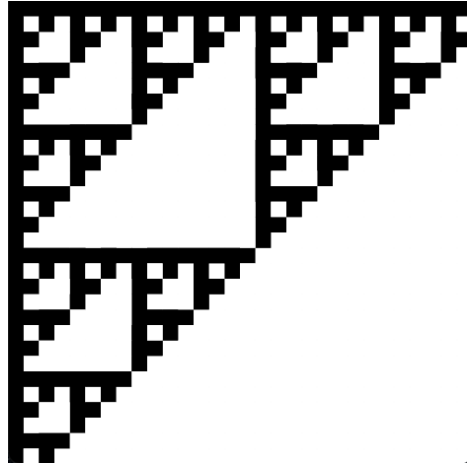


Fig. 9: The Sierpiński triangle. Target pattern for testing performance on complex patterns.

Figure 10 shows the growth dynamics of the best evolved rule for each method on a 30×30 grid. All three methods discover a local rule that produces the correct substructure: by $t = 31$, every method renders an exact Sierpiński triangle and

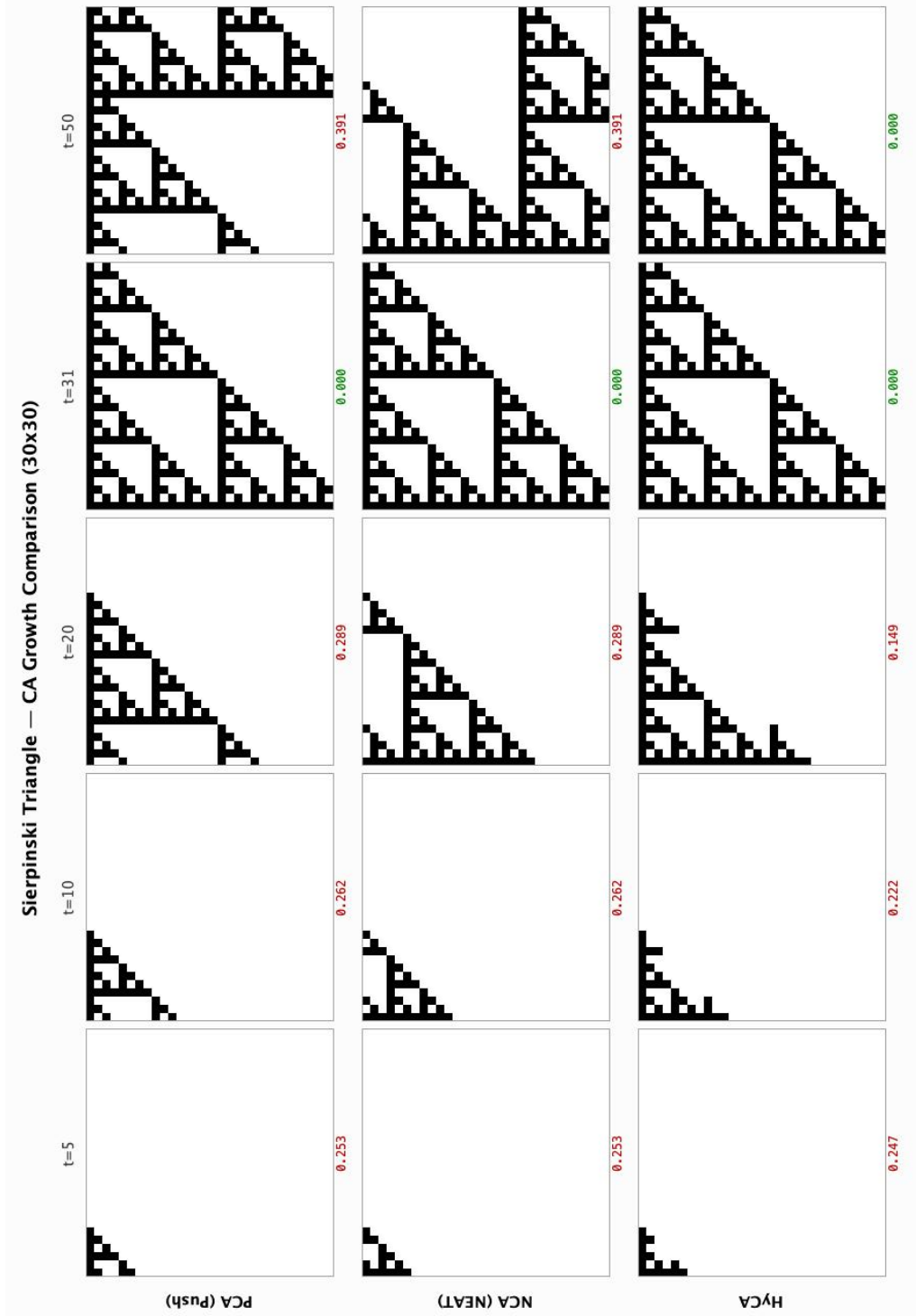


Fig. 10: Sierpinski growth across CA architectures. HyCA learns the local construction rule and stabilizes once the fractal forms. NCA and PCA reproduce the correct local substructure but add new layers on a fixed edge causing the entire pattern to drift across the grid rather than stabilize.

reaches zero error. In subsequent time steps, the methods diverge. The HyCA stabilizes once the fractal forms and maintains zero error through $t = 50$. By contrast, NCA and PCA, do not recognize the structure at $t = 31$ as the fixed target point. Instead, they continue to apply their evolved update rule. By $t = 50$ both have recovered an error of 0.391.

A second interesting aspect to note is how the rule belonging to each genome representation constructs the triangle. HyCA correct applies the xor methodology building the fractal from the seed point; however, both PCA and NCA add new layers on a fixed edge (PCA on the left and NCA on the top). This contributes to NCA and PCA’s tendency to continue drifting the pattern and not stabilizing.

PCA and NCA have learned the local construction aspect of the rule but not its stability aspect, while HyCA has learned both. We suspect that the CPPN component of HyCA provides a continuous signal containing a notion of which cells are inside the pattern versus along the edge that the Push program can use to gate further growth. By contrast neither a Push program over raw neighborhood values nor a CPPN alone seemingly reliably generate the same notion.

5 Rule Level Analysis

The convergence and complexity results from Section 4 establish that the three representations, PCA, NCA, and HyCA, achieve different errors levels, convergence rates, and different sized genomes, yet they say very little about the evolved rules themselves. In this section we aim to explore the properties of rules evolved by certain representations. When PCA and HyCA both reach near-zero error on the various patterns, are they finding the same local rule by two different routes, or two different rules that happen to agree. When NCA gets stuck in a higher-error plateau, is its population converging on a single rule, or spreading across many rules of comparable quality?

Fundamentally does the genome representation result in certain biases on the kinds of rules evolution can easily reach? A Push program is built from discrete Boolean branches and stack operations, therefore, may be expected to favor sparse rules that depend heavily on a few inputs. A NEAT-evolved network combines weighted sums and activations, therefore, may be expected to favor rules that depend evenly on all inputs.

To examine the bias of each genome representation we examine the evolved rules exactly. For a Von Neumann neighborhood over a binary search space, there are $2^5 = 32$ possible neighborhoods, and all rules are fully defined over such neighborhood. Consequentially, we compute a truth table for each evolved rule, mapping each possible input configuration to an output value. Table 1 shows an example.

Table 1: Truth tables for **sparse-dots** (run 0). Approaches disagree on 30/32 inputs.

Neighborhood	PCA	NCA	HyCA
	1	0	0
	1	0	0
	1	1	1
	1	1	1
	0	1	1
	0	1	1
	0	1	1
	0	1	1
	0	0	1
	0	0	1
	0	1	1
	0	1	1
	0	1	0
	0	1	0
	0	1	1
	0	1	1
	0	1	1
	0	1	1
	1	1	0
	1	1	0
	1	0	1
	1	0	1
	0	1	1
	0	1	1
	1	0	1
	1	0	1
	0	1	1
	0	1	1
	1	0	1
	1	0	1
	0	1	0
	0	1	0

5.1 Evolved Rule Similarity Across Representations

To compare the similarity of the evolved rules across genome representations we compute the Hamming distance [1] between truth tables. Given two 32-entry truth tables, their Hamming distance is the number of input configurations on which the two rules disagree, ranging from 0 (identical rules) to 32 (rules that disagree everywhere).

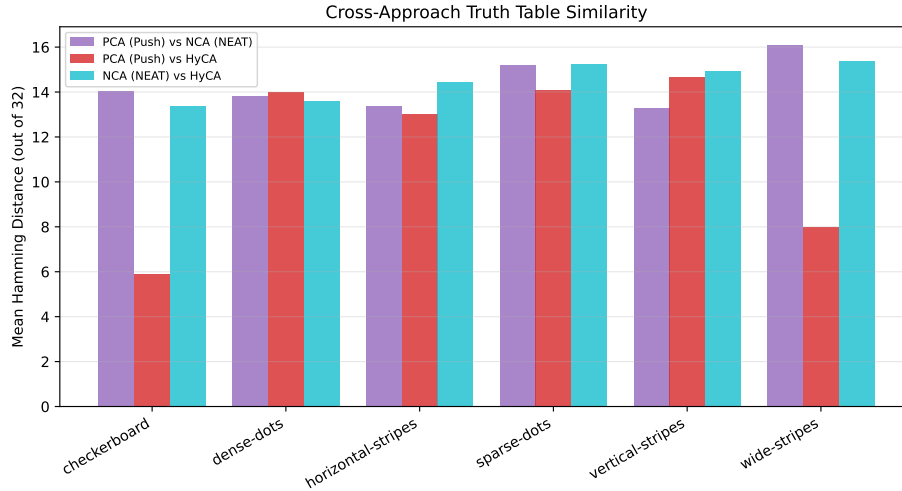


Fig. 11: Mean Hamming distance between the truth tables produced by each pair of representations, averaged across run-pairs. A distance of 0 corresponds to identical rules.

For each pair of methods and each target, Figure 11 reports the mean pairwise Hamming distance computed across all run-pairs. On the majority of targets the three pairwise distances are grouped between 13 and 17. The range expected for two rules drawn independently and uniformly from the size 32 rule space is centered at 16 with a standard deviation of roughly 2.83, since each of the 32 entries disagrees with probability 0.5. The observed pairwise distances therefore lie within this no relationship baseline.

This does not mean the evolved rules are random. Each rule is shaped by selection pressure toward low error, and the set of input configurations actually encountered during the cellular automata steps during fitness calculation. From a fixed initial seed iterated for a bounded number of update steps, only a subset of the 32 possible neighborhoods is ever visited. The rule’s behavior of the remaining configurations has no effect on fitness. The visited subset is constrained by selection, while the unvisited subset is effectively free and computed by the rule. For PCA this is the evolved construction set; NCA is the evolved weight and architecture configuration; and HyCA is the pair of CPPN and Push. The

high Hamming distances we observe are therefore a result of each representation exploring a different region of the unconstrained part of the rule space, not of the rules being random on the inputs that matter.

Two targets break this pattern. On checkerboard and wide-stripes, the PCA-HyCA distance drops well below the random baseline (roughly 6 and 8 respectively), while the PCA NCA and NCA-HyCA distances remain near 14–16. The asymmetry indicates that HyCA agrees with PCA far more than chance. This indicates that the Push component of HyCA is doing the majority of the discrete decision making, with the CPPN signal having little to no contribution to the final output.

5.2 Input Influence

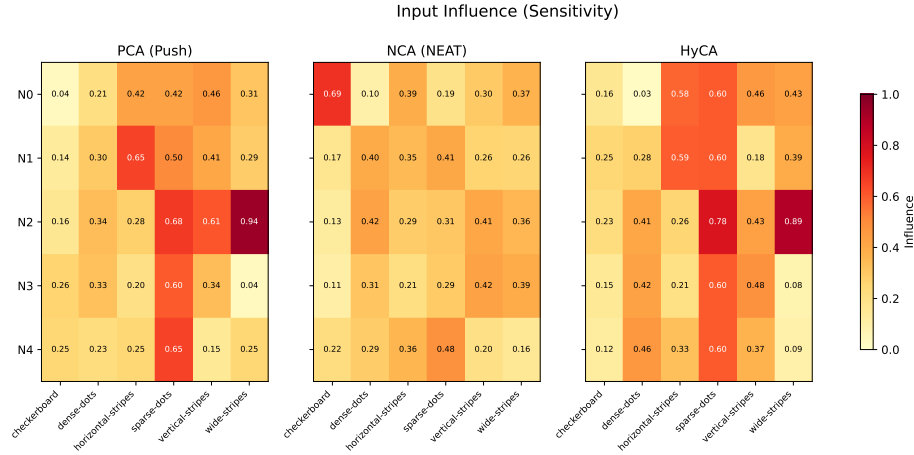


Fig. 12: Per-input influence for each representation across the six targets. Influence is the fraction of the 32 input configurations on which flipping a single neighborhood position changes the rule’s output, averaged across runs. Values near 0 indicate the rule effectively ignores that position; values near 1 indicate the rule’s output flips with that position on nearly every input.

Section 5.1 shows that the three representations explore different parts of rule space, but not what kind of rule each representation finds. To explore this we compute the input influence on each evolved rule. For each rule we measure, for every neighborhood position N_i , the fraction of the 32 input configurations on which flipping N_i changes the output. A rule that depends heavily on a single input will produced one large value and four non-zero values. A rule that depends on all five inputs will produce five roughly equal value. Figure ?? reports these influences, averaged across runs, for each representation and target.

The three genome representations produce qualitatively different results. PCA is sparse and asymmetric: on most targets one or two of the neighborhood positions carry most of the weight while the others remain near zero. The most relevant example is wide-stripes where N_2 has influence 0.94 and N_3 has 0.04, meaning the evolved rule is dependent on almost exclusively one neighbor. NCA produces an opposite pattern. Its influences are more diverse and roughly uniform, with most entries falling in the 0.2–0.4 range and only one value (checkerboard, N_0 at 0.69) standing out. The evolved networks distribute their dependence across all five inputs rather than focusing on one. HyCA follows PCA. On wide-stripes it produces $N_2 = 0.89$, $N_3 = 0.08$, $N_4 = 0.09$, mirroring PCA’s asymmetry rather than NCA’s spread. On sparse-dots the same pattern holds, with HyCA and PCA both showing elevated influence on N_2 – N_4 while NCA stays moderate.

These input influence results are consistent with the nature of each genome. Push programs are built from discrete Boolean branches and stack operations. A NEAT-evolved network, by contract, compute weighted sums followed by activations, and from a random initialization every input is attached to some weight, which effects the output. Then during evolution producing a zero-influence input requires reducing the weight to zero, which is more difficult than remaining small. PCA therefore tends toward rules that are sharply position specific, NCA toward rules that are smoothly position-distributed, and HyCA, whose decision is ultimately made by the Push component, inherits PCA’s sparse signature.

5.3 Generalization to Novel Initial Conditions

The previous rule analyses have characterized the evolved rules based on their truth tables and the structural biases of each representation. An additional question is how well the evolved rules generalize to unseen conditions. Each rule was evolved from a fixed initial condition, a single live cell at the grid origin, and the convergence results in Section 4.2 measure how well that specific trajectory reproduces the target. The convergence error does not test if the rule has learned the target as a true local rule rather, reachable from any starting condition. To assess this, we re-evaluated each rule on three held-out initial conditions; a single seed at a novel grid position, a sparse 3-cell seed, and a random fill with 50% density. We changed the number of evaluation steps to 100; however, the grid size remained the same.

Figure 13 reports the mean error across runs for each method and each evaluation condition. The training condition is at or near zero error for every method on every target, as expected from as expected Section 4.2. The three held-out conditions jump to roughly 0.3–0.5 across the board, with PCA, NCA, and HyCA being largely indistinguishable withing targets. Whatever the genome representation, the evolved rule seems to produce a trajectory of update steps that happens to pass through the target pattern at the evaluated time step, but fails to produce the target from other starting configurations.

The aggregate picture changes when we look at the best-generalizing rule each method produced rather than the average. Figure 14 reports the minimum

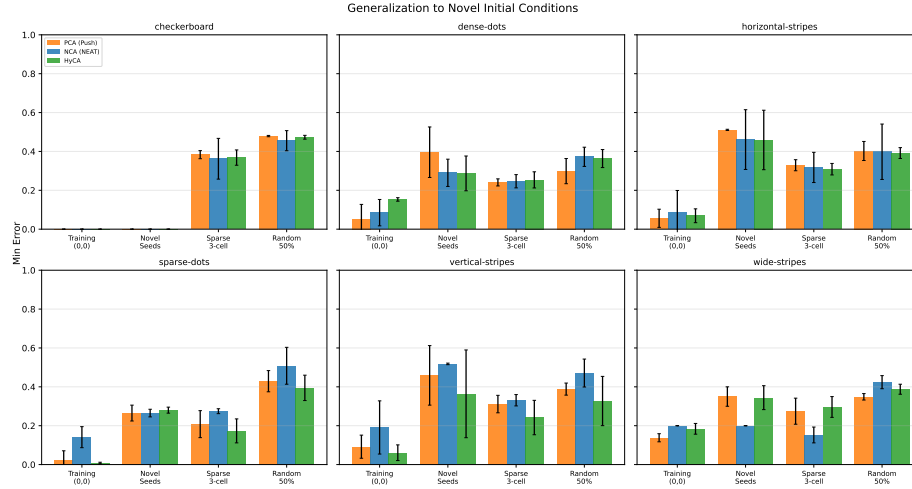


Fig. 13: Mean error across the ten runs for each (method, condition) pair, evaluated from the trained initial condition (a single seed at the origin) and from three classes of held-out initial conditions: a single seed at a novel grid position, a sparse 3-cell seed, and a random fill with 50% density. Lower is better.

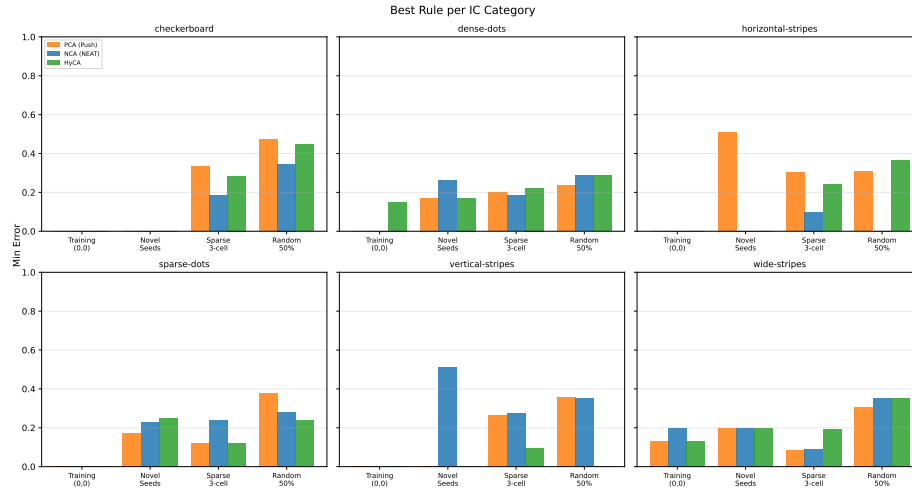


Fig. 14: Minimum error across the ten runs for each (method, condition) pair — the best-generalizing rule each method produced. A near-zero bar on a held-out condition indicates that at least one run discovered a rule that reproduces the target from that initial condition.

error across the ten runs for each (method, condition) pair. Here we see there exists rules which generalize very well to certain initial conditions. For instance in the horizontal-stripes target, we see HCA and HyCA are both able to solve the target in the training grid, and also produce a rule which finds the pattern when beginning from a novel seed. Furthermore on the same target pattern we see that NCA is able to find a rule which finds the target when beginning from a random grid of cells. Across the other target patterns, the majority of instances where a rule generalizes well are focused on the novel seed condition, with the aforementioned NCA random grid rule being the only one to generalize to another starting condition. Despite this, the non aggregate results show evidence that generalizable rules exist in the evolutionary search space, yet our experimental setup is unable to reliably find them. This suggests further investigation into a robust fitness function and selection method is needed. By evaluating each candidate rule starting from different initial conditions, and on differing grid sizes would theoretically evolve a robust rule; however, require more generations, and computational resources.

6 Discussion

tie back to evolution, ie either the evolutionary processes or Push. Something about Genome Representation? Some discussion of the biological representation and how hybrid was better

The convergence results on the six simple targets (Section 4.2) do not produce a clear winner, between PCA, NCA and HyCA. PCA reaches the lowest final error on most patterns, NCA converges quickly but plateaus on targets requiring sharp dependence on a single neighborhood position, and HyCA is competitive throughout while paying for its expressiveness with a larger combined genome. The success of different representation choices is largely a matter of fit between the genome’s natural biases and the structure of the local rule.

The Sierpiński triangle requires each method to evolve a more nuanced pattern, where we see a more defined vector (Section 4.4). All three representations discover a rule that produces the correct substructure by $t = 31$, but only HyCA stabilizes once the fractal forms; PCA and NCA continue applying their construction rule and drift to an error of 0.391 by $t = 50$. Here the main distinction is whether the method can grow the pattern and whether it can stop. This question is nonexistent on the simple periodic targets, where any correct rule is stable (either static or oscillates) under continued application.

The rule-level analyses help explain why and reveal a more nuanced issue with evolution. The hamming distance and input-influence results show that the three representations occupy very different regions of the rule space. PCA is boolean dependent, resulting in a harsher more disjoint rule space. Conversely, NCA can explore a more continuous space, finding structurally different rules. This is reflected in the analysis of the rules, with PCA and HyCA producing position specific rules and NCA distributing influence across all five inputs. However, the generalization experiments reveal an additional trend between the rules and

the evolutionary process. We see in the generalization to various initial conditions that in general all three genome representations produce lackluster results. Rules with near-zero training error, fail badly when run from different starting conditions. This shows that the evolved rules are not true local update rules and are instead rules which map an initial seed to a pattern, along a trajectory found during evolution.

If we revisit the results when evolving the Sierpiński triangle (Section 4.4) the result becomes more meaningful in light of this trajectory-following behavior. The simple-target differences and representation-level biases are real, but they exist in an environment where any of the three methods can find a serviceable trajectory from the training seed to a near-target state. In contrast the Sierpiński triangle requires construction of a non-trivial pattern from a single seed and then recognizing the resulting configuration as a fixed point, from only a five-cell binary neighborhood with no positional input. HyCA is the only representation in our experiments that achieves this. We hypothesize the difference is due to the CPPN providing a continuous signal that the Push program can threshold against to gate further updates, separating the construction logic from the stability check in a way that neither component achieves alone. Among the three representations we study, HyCA is the only one that, from purely local information, learns both to build and to preserve a complex pattern.

6.1 Limitations

One of the bigger limitations of our investigation is our focus on simple patterns. This focus did enable us to gain some traction in exploring multiple different approaches, whereas more difficult patterns (such as arbitrary bitmaps with less symmetry and uniformity) drastically increase the problem difficulty and would have made it hard to make progress. But the simplicity of the patterns we studied also limits what we can say in general. We were initially interested in morphogenesis and self-repair, but self-repair is most interesting in cases of non-uniformity; we wonder how a damaged arm knows to regrow as an arm but a damaged nose knows to regrow as a nose, or even how a heterogeneous body with distinct components (arms, legs, internal organs, etc.) can grow from a single (and thus homogeneous) cell. In contrast, self-repair of a very homogeneous, a uniform pattern like a checkerboard is not as hard to imagine and not as illuminating.

6.2 Future Work

Future research should focus on moving beyond highly symmetric patterns to evaluate how these genome representations scale to complex, heterogeneous bitmaps. While our current study established a baseline using uniform patterns like checkerboards, the true test of PCA and NCA lies in their ability to govern the morphogenesis of asymmetric structures. To facilitate this, we propose extending the PCA framework to include positional encoding (x, y) , mirroring the inputs used in CA-NEAT. Furthermore, to evolve true local update rules rather than

specific growth trajectories, future fitness functions should evaluate candidate rules across multiple initial conditions and varying grid sizes, ensuring that the discovered behaviors are robust to noise and perturbation.

Building on the unique success of the Hybrid Cellular Automata (HyCA) in stabilizing fractal growth, there is significant potential in exploring more sophisticated hybrid architectures. We hypothesize that using a neural network as a "gating" mechanism to selectively activate specific Push programs could introduce modularity and functional reuse into the CA framework, much like external chemical signals regulate gene networks in biological cells.

Finally, a targeted analysis of "unvisited" states within the evolved truth tables is needed. By understanding how different representations handle input configurations that fall outside the typical training trajectory, we can better design evolutionary systems that achieve the robust, self-repairing capabilities observed in natural biological systems.

References

1. Hamming, R.W.: Error detecting and error correcting codes. *The Bell System Technical Journal* **29**(2), 147–160 (1950). <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>
2. Helmuth, T., McPhee, N.F., Spector, L.: Program synthesis using uniform mutation by addition and deletion. In: *Proceedings of the Genetic and Evolutionary Computation Conference*. p. 1127–1134. GECCO '18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3205455.3205603>, <https://doi.org/10.1145/3205455.3205603>
3. Krishna, V.M., Mishra, S., Gurralla, J.: Cellular automata and their applications: A review. *International Journal of Computer Applications* **184**(5), 13–17 (2022)
4. Mordvintsev, A., Randazzo, E., Niklasson, E., Levin, M.: Growing neural cellular automata. *Distill* (2020). <https://doi.org/10.23915/distill.00023>
5. Nichele, S., Ose, M.B., Risi, S., Tufte, G.: CA-NEAT: Evolved compositional pattern producing networks for cellular automata morphogenesis and replication. *IEEE Transactions on Cognitive and Developmental Systems* **10**(3), 687–700 (2018). <https://doi.org/10.1109/TCDS.2017.2737082>
6. Spector, L., Robinson, A.: Genetic Programming and Autoconstructive Evolution with the Push Programming Language. *Genetic Programming and Evolvable Machines* **3**(1), 7–40 (Mar 2002). <https://doi.org/10.1023/A:1014538503543>, <https://doi.org/10.1023/A:1014538503543>
7. Stanley, K.O.: Compositional pattern producing networks: A novel abstraction of development. *Genetic Programming and Evolvable Machines* **8**(2), 131–162 (2007)
8. Stanley, K.O., Miikkulainen, R.: Evolving neural networks through augmenting topologies. *Evolutionary Computation* **10**(2), 99–127 (2002)